Input: (375, 1242, 3)

raw_img = cv2.read (file path)

print (raw_img) = [[[18 22 17] ..... [37 30 22]]]
[28 24 16] [38 29 22]
[33 29 25] [35 30 20]

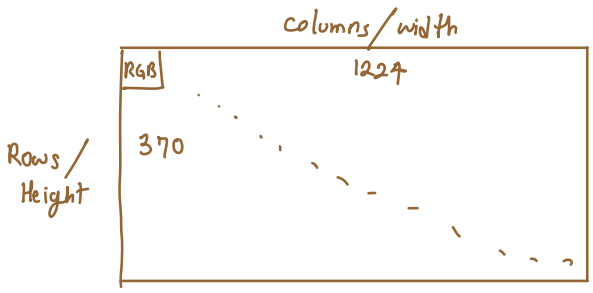
RGB

print (raw_img.shape) = (370, 1224, 3)

Rows/Height ↓ RGB. channels

Columns/width

we normalize RGB values to 0~1 b/c 0~255 has wide range and sharp change ex.) 200 → 255 would confuse the program.



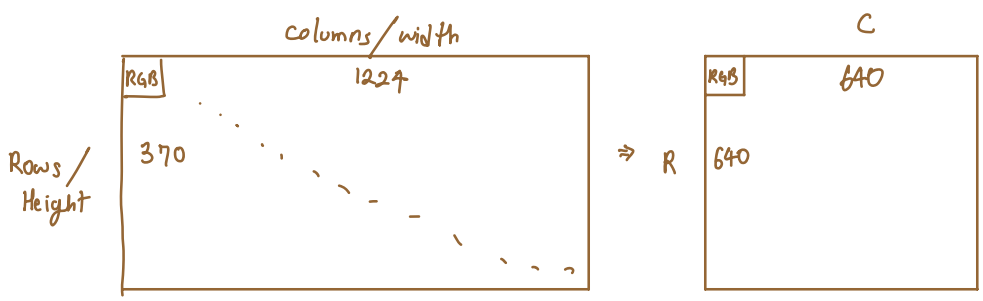
2.) Resize. KITTI image → resized image. using Interpolation

↳ type of estimation, a method of constructing new data points based on the range of discrete set of known data points.

(375, 1242, 3) → (640, 640, 3)

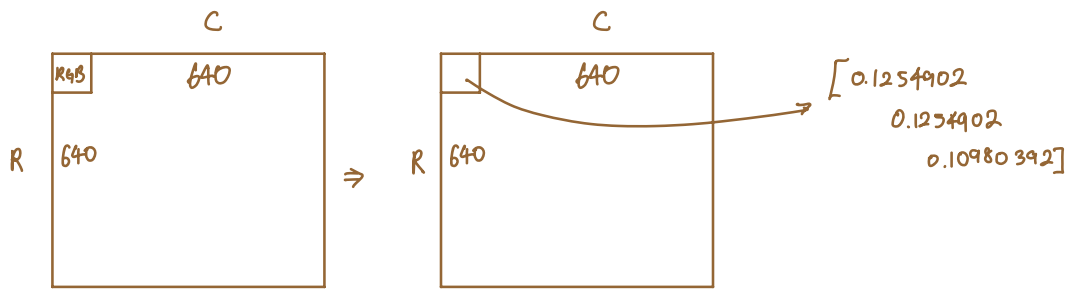
resized_img = cv2.resize (raw_img , (640, 640))

↑ width , ↑ height.



2.) Normalization

resized_img → normalized_img



3.) Tensor

A.) CPU vs GPU

1.) CPU (Central processing unit)
- versatile, few-core processor for sequential tasks and system management.
  ex.) running operating systems

2.) GPU (Graphics Processing Unit)
- thousands of tiny cores that can do millions of math problems at the exact same time.
  ex.) massive parallel processing, excelling at graphics, AI...

↳ so why is this important?

1.) - PyTorch is designed to live on <u>GPU</u>. Numpy array doesn't have the "wiring" to send data to GPU.

2.) The "Memory" of Math (Autograd)

- Numpy is "forgetful". $x = 5 \times 2$, gives you 10 and forget <u>how</u> it got there.

- PyTorch Tensors have a "memory" (called Computational Graph). They remember every math operation performed on them.

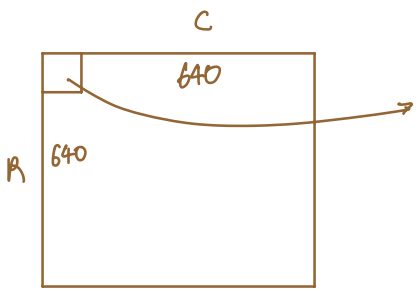
  ↳ when AI makes mistakes during training, it needs to go <u>backward</u> through all the math to figure out which "knob" to fix it. Because Tensors track their own history, it can automatically calculate the "slope" (gradient) of the error.

3.) Shared memory

- might concern "does converting 100 images from NumPy to Tensor take up twice as much as RAM?"
  ↳ Random-Access Memory : temporary data storage and instructions for active applications.

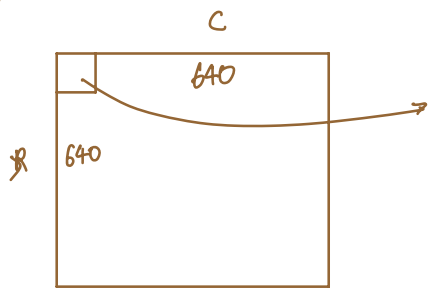
  - No : PyTorch doesn't copy the data in RAM, it just points to the same memory address and puts "PyTorch label" on it.

① tensor_img = torch.from_numpy(normalized_img)



[0.1254902
0.1254902
0.10980392]

⇓ only the type has changed from NumPy array to PyTorch Tensor.



[0.1254902
0.1254902
0.10980392]

② tensor_img = torch.from_numpy(normalized_img).permute(2,0,1)

Numpy type([640,640,3]) → Tensor type([640,640,3]) → Tensor type([3,640,640])

original shape : (640, 640, 3)
0 1 2

.permute(2,0,1) : (2, 0, 1)

why?

- AI-Friendly : AI treats colors like "layers" or "planes"
  - it wants to look at the entire "Red Layer"

then the "Green Layer"
then the "Blue Layer"

this enables AI to effectively "slide" its filters accross planes independently.

4.) Batch

1. why?
   - imagine that you are a teacher grading exams. You could grade them one by one, but it would be faster to stack multiple exams together, look at first question for all of them, then the next question for all and so on.

- In AI, parallelism is ~~king~~ key. A GPU is designed to do same math on multiple images at the exact same time.
  - If the Batch is 1, the GPU processes 1 image.
  - If the Batch is 16, " " " 16 images in almost the same amount of time it takes for 1.

↳ Due to this, YOLO v8 built the "input gate" of the model to always look for a 4D container:

[Batch size, Channels, Height, width]

```
final_input = tensor_img.unsqueeze(0)
```

↓

the word "squeeze" in math means to get rid of dimension. "Unsqueeze" would add one.

create a new "wrapper" dimension at Index 0.

→ does not change a single pixel or a single color value. It just places your image inside a "list of images" that happens to be, in this example, to only have 1 item in it.

Input → Backbone

before making our Backbone, need to define labels for the backbone.

Feature	KITTI Language (The "text")	YOLO Language (what AI works with)
Class	written words (eg. "car", "pedestrian)	Numbers (eg. 0, 1)
Coordinates	Pixels (x_{min}, y_{min}, x_{max}, y_{max})	Percentages (x_{center}, y_{center}, width, height)
Extra Info	15 columns (Rotation, Truncation, etc)	5 columns (Box and the class)

inside label_2_first 100.xxxxxxtxt

ex.) Car 0.00 -1.57 599.41 156.40 629.75 189.25 ..... -1.56

Column 0 1 2 3 4 5 6 15

what we need : 5 columns for YOLOv8.

1.) class 2~5.) x_{min}, y_{min}, x_{max}, y_{max}

⇓

[Class ID] [x_center] [y_center] [width] [height]

Step A.) Width & Height

$$width = x_{max} - x_{min}$$

$$height = y_{max} - y_{min}$$

Step B.) Center point (x, y)

$$x_{center} = x_{min} + \frac{width}{2}$$

$$y_{center} = y_{min} + \frac{height}{2}$$

Step C.) Normalize x_{center} y_{center} W H